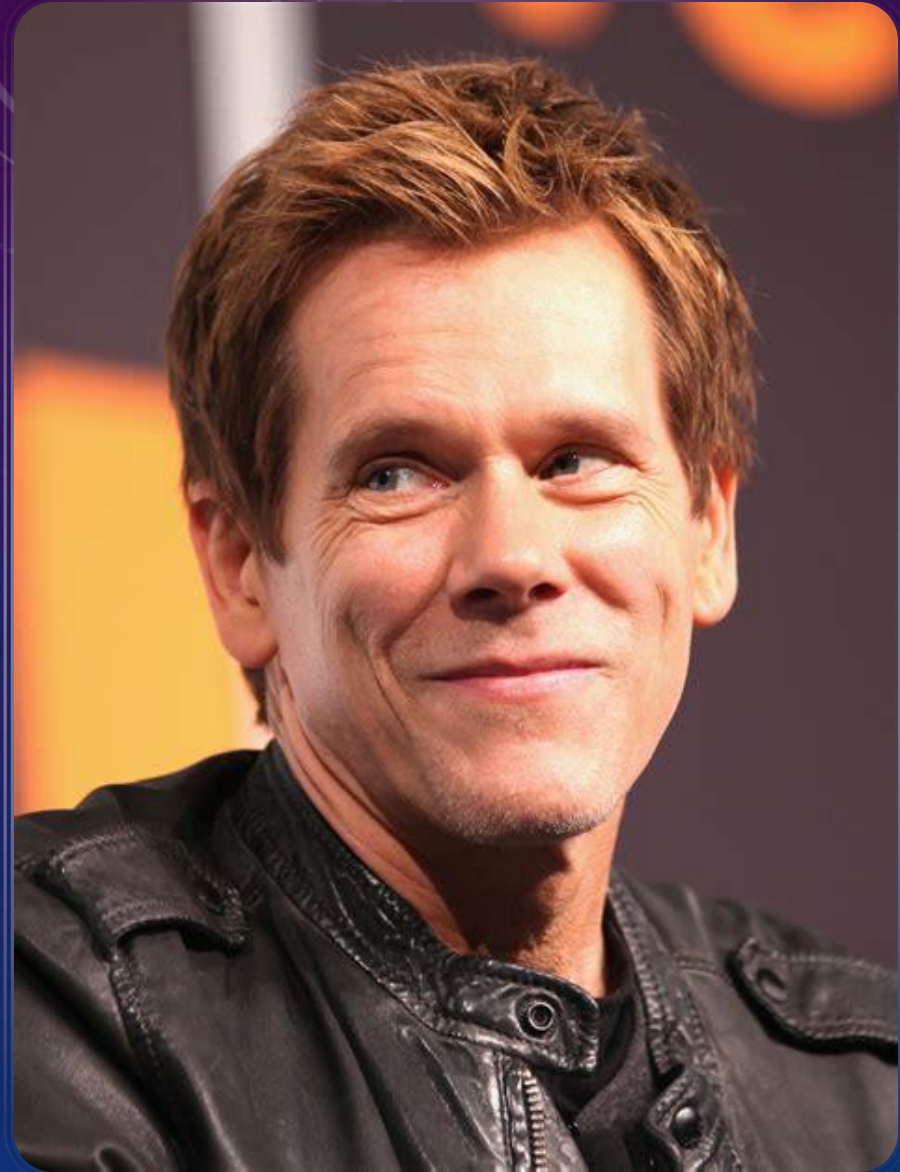


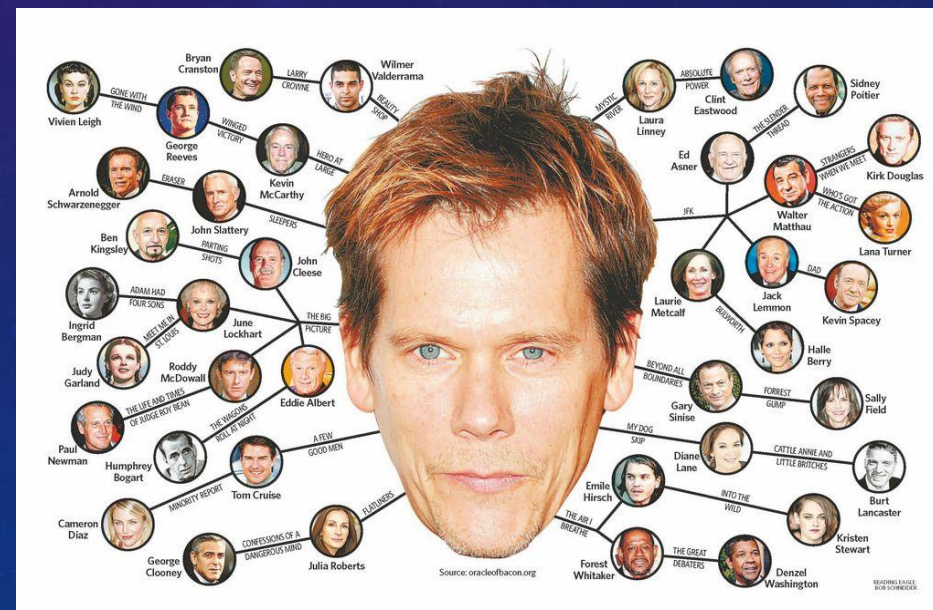
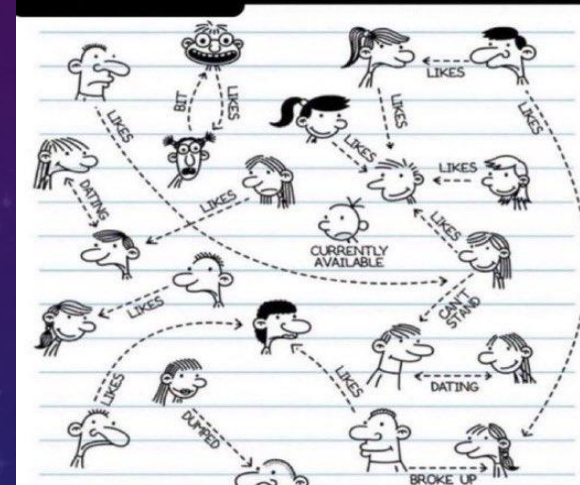
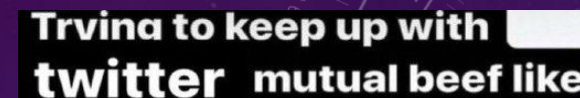
SIX DEGREES

PHILIPP SHCHETININ, KALEN HAZLETT, MORGAN
QUACKENBUSH



OBJECTIVE

- Create a program that finds minimum separation between users based on mutual connection
 - "Mutual" defined as someone a user follows and is followed by.
- Visualize the graph of users with their connections



PROJECT FLOW

- Rust prompts the user to enter in two usernames
- Rust then sends a username to the mutuals checker via command line.
- Python parses database file and returns intersection of followers and following
- Process repeats until graph is mapped out, then BFS finds the shortest path.
- Data is sent to the visualizer, which plots the formed graph.



TWO LANGUAGES?

- Originally Going to Web Scrape
 - Twitter API
- Good Data Science Libraries in Python
- Rust is emerging in critical systems and is compiled
--> faster
 - Has 'memory safety'
- Twitter API is expensive, and we are broke, so we created our own database



DATABASE

- Collection of tables (followers and following) that can search itself
- Contains an index that organizes the tables into B-Trees, making searching much faster ($O(\log n)$)
- Without the index, it would be no different than a line-by-line search $O(n)$

```

1 SQLite format 3 NULDLENULSOHSOHNUL@ NULNULNULACKNULNULBELNULNULNULNULNULNULNULNULNULNULENENQNULN
2 SI ? NULACK SO 0 NUL SI ? SI ? SI 6 SO ? SO ? SO 0 NULNULNULNULNULNULNULNULNULNULNULNULNULNULNULNULNULN
3 NULNULNULSUB SI ~ NUL SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI
4 STX SI Metx FF STX SI Letx VTSTX SI Ketx
5 STX SI JETx      STX SI Ietx BS STX SI HetxBELSTX SI GETXACKSTX SI FETXENQSTX SI EETXEOTSTX SI DETXETXSTX SI CETXST
6 NULNULNULSUB SI e NUL SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI
7 ENQETX SI SOHL FF ENQETX SI SOHK V TENQETX SI SOHJ
8 ENQETX SI SOHI   ENQETX SI SOHH BS ENQETX SI SOHG BELENGETX SI SOHF ACKENGETX SI SOHE ENQENGETX SI SOHD EOTENGETX S
9 NULNULNUL?
10 W NUL SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ? SI ?
11 ?
12 ?
13 ?
14 ?
15 ?
16 ?
17 ?
18 ?
19 ?
20 ?

```

```

1  A:
2      followers - G M Z
3      following - C E H O U Z
4
5  B:
6      followers - S R T N I Q
7      following - E K U W X
8
9  C:
10     followers - U Z V Y P A R G H Q O L I D F W K X E N J S
11     following - D E I J O P R S U X
12
13  D:
14     followers - N E C G J M L Q
15     following - C E G I M O T Z
16
17  E:
18     followers - C Q K A B T S G D
19     following - C D J O U W Y
20

```

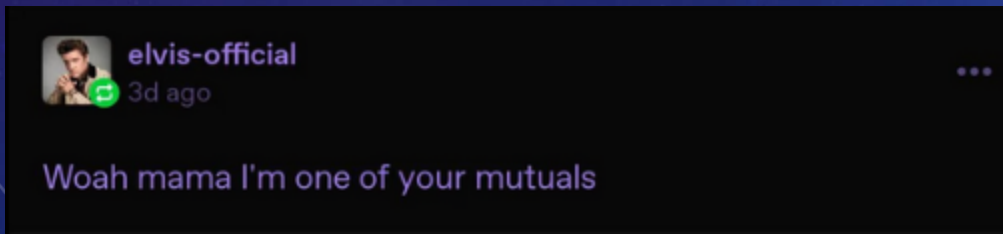
```

1 A:G,M,Z|C,E,H,O,U,Z
2 B:S,R,T,N,I,Q|E,K,U,W,X
3 C:U,Z,V,Y,P,A,R,G,H,Q,O,L,I,D,F,W,K,X,E,N,J,S|D,E,I,J,O,P,R,S,U,X
4 D:N,E,C,G,J,M,L,Q|C,E,G,I,M,O,T,Z
5 E:C,Q,K,A,B,T,S,G,D|C,D,J,O,U,W,Y
6 F:J,K,L|C,G,H,J,K,O,S,U,X,Z
7 G:K,V,J,Q,F,D,N,L|A,C,D,E,H,S,X,Z
8 H:U,V,A,W,F,S,K,G,R,X,L|C,M,N,O,R,T,X,Y
9 I:T,D,V,K,C,P,J|B,C,O,U,Y
10 J:K,P,F,C,E|C,D,F,G,I,N,P,R,S
11 K:F,N,M,L,B,O,T,X|C,E,F,G,H,I,J,N,P,Q,V,W,X
12 L:X,Q,V,U|C,D,F,G,H,K,O,P,T,W,Y
13 M:N,U,Q,W,H,D|A,D,K,P,W,X

```

MUTUAL SORT

- Receives an account that needs its mutuals found.
- Runs an SQL Query that looks in the followers table and following table and collects the rows relevant to the user.
- Extracts the data (the users followers and who the user is following) and puts that extracted data into sets.
- Finds the intersection of the two sets and sends it back through command line



```
for line in sys.stdin:
    username = line.strip()
    # reads in data from command line strips any white space

    followers, following = find_user(username)
    #sends data to find_user, puts that data into two different variables

    if followers is None:
        print(json.dumps({"username": username, "error": "not found"}))
        sys.stdout.flush()
        continue
    #if the data doesnt exist I.E. the username input wasnt found, returns an error to rust and flushes

    mutuals = followers & following
    #Finds the intersection of the two sets

    mutuals = sorted(mutuals)
    #Sorts the intersection
```

```
# Locate the DB file relative to this script
base_dir = os.path.dirname(os.path.abspath(__file__)) #gets absolute path of mutuals.py
db_path = os.path.join(base_dir, db_name) #creates a path to the database

# Connect to the SQLite database
conn = sqlite3.connect(db_path) #opens a connection to the db and connection object
cur = conn.cursor() #creates a cursor to execute commands

# Fetch followers
cur.execute("SELECT follower FROM followers WHERE user = ?", (fusername,)) #Looks in table followers, uses ? to safely sub username
followers = {row[0] for row in cur.fetchall()} #extracts the username from the tuple holding data

# Fetch following
cur.execute("SELECT follows FROM following WHERE user = ?", (fusername,))
following = {row[0] for row in cur.fetchall()}

conn.close() #closes connection to sql

# If both empty, treat user as "not found" for output consistency
if not followers and not following:
    return None, None

return followers, following
```

RUST

- Main data structure of the project
 - Launches the python script as a sub-process
 - Has the graph represented as an adjacency list
 - Computes BFS traversal to find shortest/only path
 - Trends towards $O(V+E)$ where V is the number of Vertices and E is the number of Edges
 - Did not use Dijkstra's because the weight was not important



```
//spawn the python script as a sub process
let mut child: Child = Command::new(program: "python3") Command
    .arg("-u") &mut Command
    .arg("py_script_2.py") &mut Command//change for actual scri
    .stdin(cfg: Stdio::piped()) &mut Command
    .stdout(cfg: Stdio::piped()) &mut Command
    .spawn()?;
```

GRAPH

```
pub struct Graph<N, W = ()>{
    nodes: Vec<N>, //creates the nodes to
    adj: Vec<Vec<(NodeId,W)>> //creating the list (neighbor, weight)
}

//methods to work on the structure above. Like Classes in Python or objects in C++
impl<N, W> Graph<N, W>{
    //public function: create a new node
    pub fn new() -> Self{
        //create a new node using the vector library imported at top Vectors are lists that can change size safely
        Self { nodes: Vec::new(), adj: Vec::new()}
    }

    //function to add a new node to the graph (&mut means a parameter is going to be a mutable variable)
    pub fn add_node(&mut self, data: N) -> NodeId{
        let id = self.nodes.len(); //creates a variable of useize

        self.nodes.push(data);
        self.adj.push(Vec::new());

        //returns the id
        id
    }
}
```

COMMUNICATION

```
//spawn the python script as a sub process
let mut child: Child = Command::new(program: "python3") Command
    .arg("-u") &mut Command
    .arg("py_script_2.py") &mut Command//change for actual script name. Use .env file if needed/want
    .stdin(cfg: Stdio::piped()) &mut Command
    .stdout(cfg: Stdio::piped()) &mut Command
    .spawn()?;

//creating variables for data
let mut py_stdin: ChildStdin = child.stdin.take().expect(msg: "failed to open python stdin");
let py_stdout: ChildStdout = child.stdout.take().expect(msg: "failed to open python stdout");
let mut reader: BufReader<ChildStdout> = BufReader::new(inner: py_stdout);

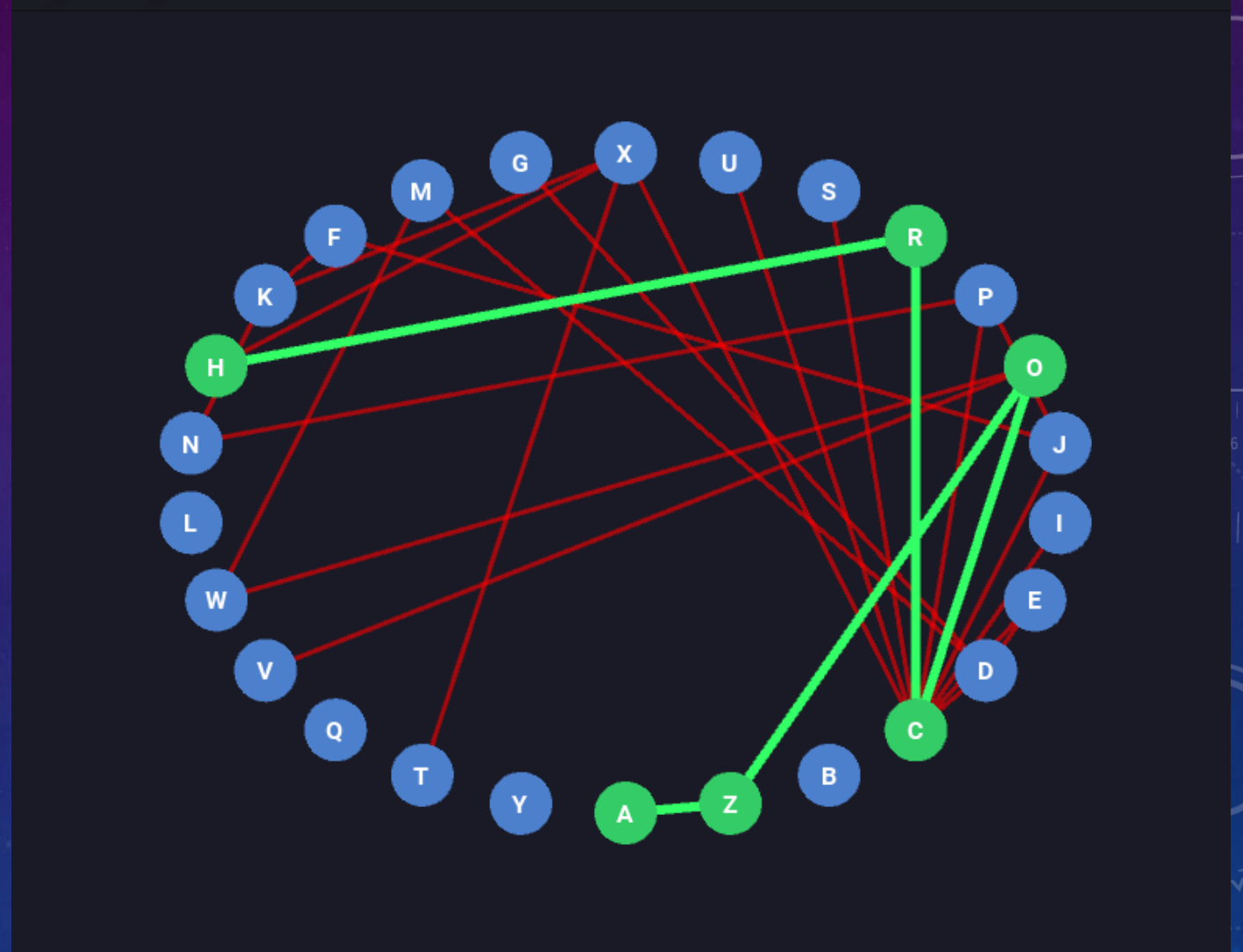
//create some variables for traversing later
let mut g: Graph<String, f64> = Graph::new();
let mut index: HashMap<String, NodeId> = HashMap::new();

//could have read in from a file or gotten input, but why? this works for the sample data we have
let usernames: Vec<&str> = vec!["a", "b", "c", "d", "e", "f", "g", "h", "i", "j",
                                "k", "l", "m", "n", "o", "p", "q", "r",
                                "s", "t", "u", "v", "w", "x", "y", "z"];
//loop through the list of usernames, parsing each one
for u: &&str in &usernames {
    //normalize the input
    let username: String = canon(u);
    // send username to the python script
    writeln!(py_stdin, "{}", username)?;
    py_stdin.flush()?; //flush the buffer

    let mut line: String = String::new();
    let n: usize = reader.read_line(buf: &mut line)?; //read the python line
```

VISUALIZATION

- Receives data from Rust code in json
 - Graph Information
 - Traversal Information
- Plots Each user
- Creates edges in between each user
- Creates the path of traversal



DEMONSTRATION

- It's a demo



CHALLENGES

- Conceptual Understanding of the Project
- Using Two languages (Using Rust)
- Finding/creating a data set that worked for our purposes
- Debugging
 - Json parsing



POTENTIAL ENCHANTMENTS

- Live separation checker
- Larger database
- Use actual API (looking for sponsors)
- Integrating Python into Rust using PyO3



RESOURCES AND AI STATEMENT

- Multiple online articles:
- <https://www.geeksforgeeks.org/dsa/adjacency-matrix/>
- <https://timclicks.dev/tip/how-to-create-a-2d-matrix-in-rust>
- <https://www.geeksforgeeks.org/c/c-program-to-implement-adjacency-list/>
- <https://medium.com/@kriangkrai.ratt/shortest-path-dijkstra-algorithm-with-rust-8fe1867d052a>
- https://www.w3schools.com/sql/sql_intro.asp
- OpenAI ChatGPT model 5.0 for debugging and language translation
- Claude AI enhanced the visual script

QUESTIONS?

